

Mobile Video Networks for Human Action Recognition

Evelina Bartosevič

Faculty of Mathematics and Informatics
Vilnius, Lithuania
evelina.bartosevic@mif.stud.vu.lt

Austėja Balkytė

Faculty of Mathematics and Informatics
Vilnius, Lithuania
austėja.balkyte@mif.stud.vu.lt

Saulius Lipkevičius

Faculty of Mathematics and Informatics
Vilnius, Lithuania
saulius.lipkevicius@mif.stud.vu.lt

Abstract—In this project A0, A1 and A2 base models of Mobile Video Networks (*MoViNets*) are trained and tested on *HMDB51* dataset for the first time. The models were discussed as they are most likely to be used on mobile devices compared to the other existing *MoViNets* models A3, A4 and A5. A brief introduction of *MoViNets* architecture and dataset is displayed. Obtained results show high accuracy, around $\sim 80\%$, compared to other models that were trained on the same video dataset. Moreover, *MoViNets-A1* showed the best accuracy-memory ratio out of all three models.

Index Terms—Action recognition, *MoViNets*, *HMDB51* dataset

I. INTRODUCTION

Humans easily recognize and identify actions in video but automating this procedure is challenging. However, each year new models of action recognition are created. The main task in these models is to identify the type of activity being performed in a video clip. Activity Recognition is an important problem with many societal applications including smart surveillance, video search/retrieval, intelligent robots, and other monitoring systems.

Videos are tricky for machines to handle. Their dynamic nature, as opposed to an image's static one, can make it complex for a data scientist to build models. However, we can say that videos are a collection of a set of images arranged in a specific order. These sets of images are also referred to as frames. [6]

In this study three base models of Mobile Video Networks (*MoViNets*) for efficient video recognition are used. They were first presented on April, 2021 and only trained on *Kinetics* dataset. We used the *HMDB51* dataset. Results received with the dataset applying various models are compared throughout the years, presented in Figure 1.

Motivation to use *HMDB51* dataset came from lack of papers employing *MoViNets* for Action Recognition with the mentioned data.

The rest of this paper is organized as follows: in section II we made a review of *MoViNets* models, section III describes chosen data of human actions and section IV briefly explains the results obtained throughout this work. Finally, the conclusions are given in the last section V.

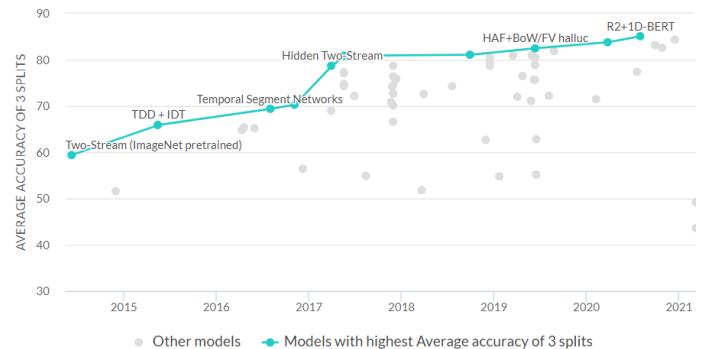


Figure 1 – Models with highest average accuracy on *HMDB51*. [1]

II. METHODS

According to [4], *MoViNets* is a family of computation and memory efficient video networks that can operate on streaming video for online inference. Model uses Neural Architecture Search (NAS) to generate efficient mobile architectures, ranging from *MobileNet*-sized models to *3D ResNet*-sized models, i.e., *MoViNet-A0* to *MoViNet-A6*. Furthermore, the usage of Causal Convolutions in a Stream Buffer allow *MoViNets* to operate on streaming video with significantly less memory usage. Base models implement standard 3D convolutions without stream buffers while streaming models implement causal 3D convolutions with stream buffers.

A. Three main steps

3D convolutional neural networks (CNNs) are accurate at video recognition but require large computation and memory budgets and do not support online inference, making them difficult to work on mobile devices. Therefore, *MoViNets* three-step approach to improve computational efficiency while substantially reducing the peak memory usage of 3D CNNs:

- 1) A video network search space is designed and neural architecture search to generate efficient and diverse 3D CNN architectures are employed.
- 2) The Stream Buffer technique that decouples memory from video clip duration are introduced, allowing 3D CNNs to embed arbitrary-length streaming video se-

quences for both training and inference with a small constant memory footprint

- 3) A simple ensembling technique is proposed to improve accuracy further without sacrificing efficiency.

B. Architecture of models

In this subsection architecture of three base models is compared, *MoViNets A0, A1*, and *A2* represent the fastest models that would most realistically be used on mobile devices.

Most notably, the network prefers large bottleneck width multipliers in the range $[2.5, 3.5]$, often expanding or shrinking them after each layer. Network prefers balanced $3 \times 3 \times 3$ kernels, except at the first down-sampling layers in the later blocks, which have $5 \times 3 \times 3$ kernels. The final stage almost exclusively uses spatial kernels of size $1 \times 5 \times 5$, indicating that high-level features for classification benefit from mostly spatial features.

There are more significant margins between larger *MoViNets (A3–A6)*. As they consume lots of GPU memory in the absence of modifications, especially as the model sizes and input frames grow, in this project these models are not analysed.

C. Description of MoViNets

In *MoViNets* stream buffer is used to reduce the network’s memory consumption from linear to constant in video length. A usual way to deal with excess of memory budget is multi-clip evaluation [7]. However, in this model a stream buffer is presented to increase efficiency. Feature map F_i of buffer concatenated ($\oplus$) with the subclip along the time dimension is computed:

$$F_i = f(B_i \oplus x_i^{clip})$$

where f represents a 3D convolution and x_i^{clip} is the current subclip. When processing the next clip, we update the contents of the buffer to:

$$B_{i+1} = (B_i \oplus x_i^{clip})_{[-b:]}$$

Here $[-b:]$ are the last b frames of concatenated input. As a result, memory consumption is dependent on $O(b + T^{clip})$.

The illustrations below (Figure 2 and 3) shows how in multi-clip evaluation overlapping subclips of an input video with 3D convolutions and average the *logits* are embed. In streaming evaluation, the stream buffer are used to carry forward input features between non-overlapping subclips.

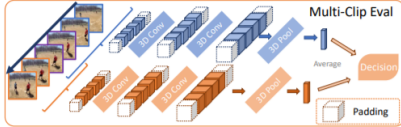


Figure 2 – Multi-clip Evaluation

Moreover, by leveraging the translation equivariant property of convolution, all temporal convolutions with Causal Convolution [5] are replaced. When using a stream buffer with *CausalConv*, causal padding can be replaced with the buffer



Figure 3 – Streaming Evaluation

itself. This helps to save memory. Architectures of all three models can be found in Appendix.

III. DATA

With nearly one billion online videos viewed everyday, an emerging new frontier in computer vision research is recognition and search in video. While much effort has been devoted to the collection and annotation of large scalable static image datasets containing thousands of image categories, human action datasets lack far behind. Here we introduce *HMDB* collected from various sources, mostly from movies, and a small proportion from public databases, such as the *Prelinger* archive, YouTube and Google videos. The dataset contains 6849 clips divided into 51 action categories, each containing a minimum of 101 clips, refer to Figure 4. The actions categories can be grouped in five types:

- **General facial actions:** smile, laugh, chew, talk.
- **Facial actions with object manipulation:** smoke, eat, drink.
- **General body movements:** cartwheel, clap hands, climb, climb stairs, dive.
- **Body movements with object interaction:** brush hair, catch.
- **Body movements for human interaction:** fencing, hug, kick someone.

In addition to the label of the action category, each clip is annotated with an action label as well as a meta-label describing the property of the clip. Based on the researcher needs there are three options of videos quality types: *good*, *medium*, *bad*. Only those video samples are rated *good* which have the quality that you can identify the single fingers during the motion. Those which do not meet this requirement are either rated *medium* or *bad* if body parts or limbs vanish while the action is executed. To add up, there are other options, for instance Video Stabilization and others. [2], [3]

Pre-processing applied in our project was as follows:

- Input was resized to have the same height and width;
- Horizontal flip was used on training set;
- Testing set was cropped randomly and center crop was used for test set;
- Number of frames was chosen 16, because it was recommended in the paper.

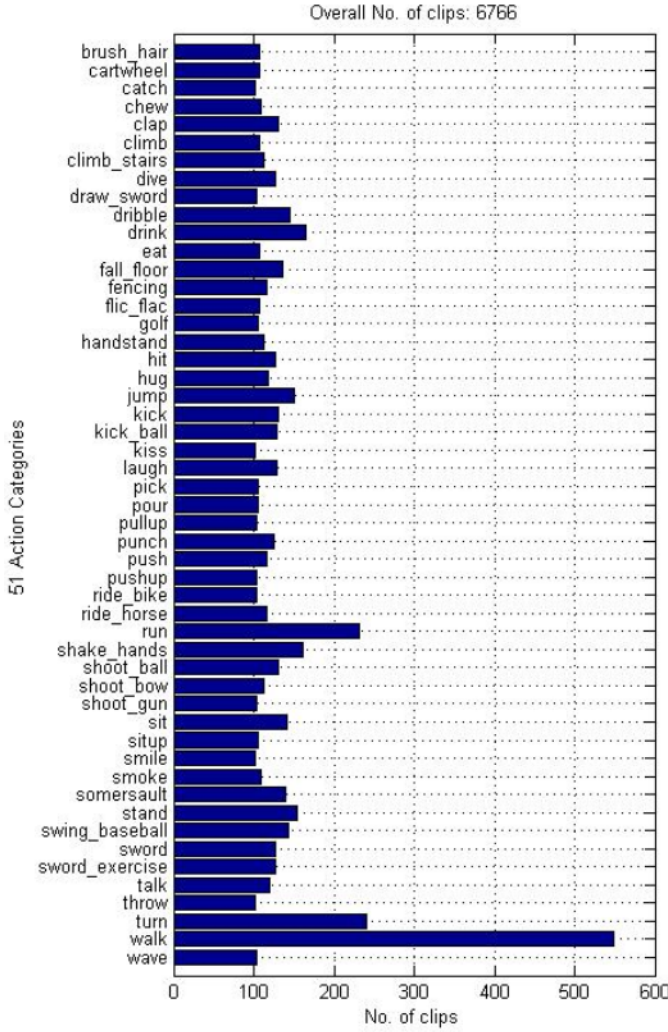


Figure 4 – The number of actions presented in the *HMDB51*

IV. RESULTS

Three Mobile Video Networks base models (pre-trained *MoViNets A0, A1*, and *A2*) were trained on all 51 action classes of the *HMDB51* train set. After training, these models are evaluated on the test set. Worth mentioning that we also tried to train *MoViNets A3–A6*, but because of the limited computational resources it was not possible. We have implemented the models described above on *Google Colab* using the *PyTorch* framework and several Python 3 libraries.

A. Training Procedure

In the beginning, we decided to train *MoViNets A0–A2* models separately with 5 epochs and batch size 16 because our dataset is large. Moreover, we used *Adam optimizer* with a learning rate = $2e - 5$ to train all 3 models and it took about 10 hours. To see, how well our models train and to avoid *over-fitting*, *validation loss* curves were plotted for each case (Figures 7, 8 and 9).

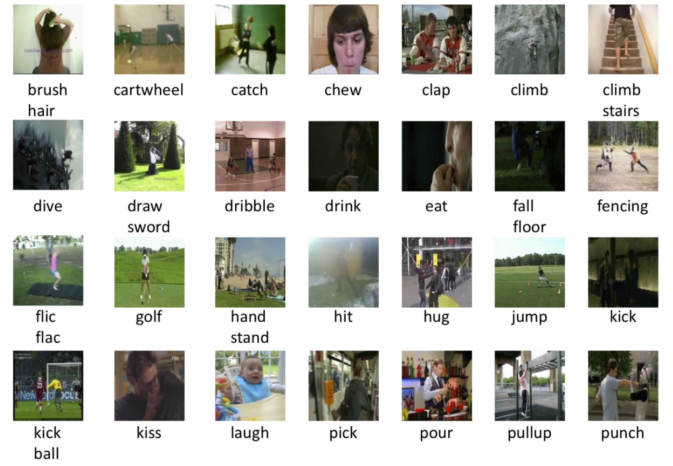


Figure 5 – *HMDB51* Visual representation. Part I

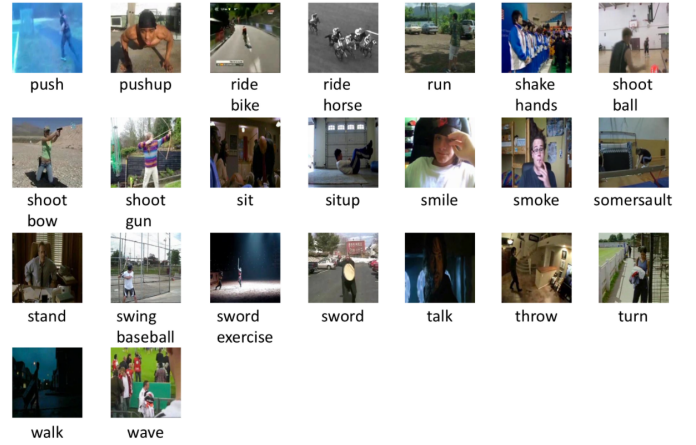


Figure 6 – *HMDB51* Visual representation. Part II

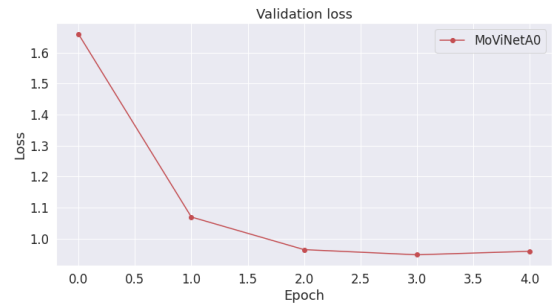


Figure 7 – *Cross-entropy* validation loss depending on number of epochs (*MoViNet-A0*).

In the *MoViNet-A0* case (Figure 7), the *validation loss* is fastly decreasing during the first 3 epochs and slowly the next 2 epochs. Therefore, we will stay with 5 epochs and there is no need to train this model for more epochs. The *cross-entropy loss* also fastly decreases in *MoViNet-A1* and *MoViNet-A2* (Figures 8 and 9) within the first 3 epochs, however, we can see that starting from the 4th epoch *validation loss* starts to increase. It means that the optimal

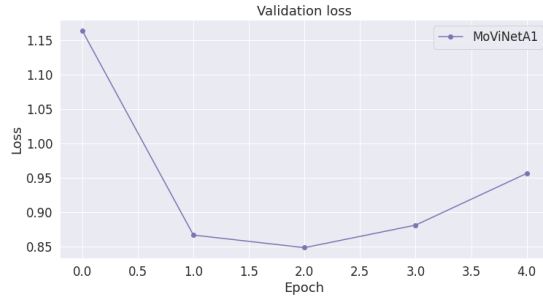


Figure 8 – *Cross-entropy* validation loss depending on number of epochs (*MoViNet-A1*).

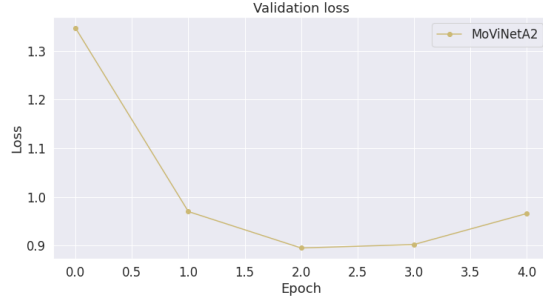


Figure 9 – *Cross-entropy* validation loss depending on number of epochs (*MoViNet-A2*).

number of epochs for these two models equal to 3 (where elbow appears).

B. Results of MoViNets

Table 1 summarizes the achieved results for all three MoViNet models. To compare the models we used three main features: accuracy, average time per epoch and GPU RAM. The first two features show the efficiency of the models and the last one was estimated because of the importance highlighted in the paper (memory usage reduction).

The first model was trained on 5 epochs and shows the worst results. While models A1 and A2 used 3 epochs and the accuracy increased significantly. However, difference in accuracy between A1 and A2 is only 0.2% when the time changed approximately 17 minutes per epoch (51 minutes on whole) and GPU RAM doubled. Therefore, the model *MoViNet-A1* might be considered as the best one.

MODEL	Accuracy	Average time per epoch	GPU RAM
MoViNet-A0	76.8%	1798s	4.9 GB
MoViNet-A1	80.9%	2057s	7 GB
MoViNet-A2	81.1%	3054s	14.9 GB

Table I – Results of *MoViNets* on *HMDB51*.

V. CONCLUSION

MoViNets models have significant accuracy on *HMDB51* dataset, what is $\sim 80\%$. The best accuracy was reached by A2, however considering other features, average training time and GPU RAM, A1 seems more suitable. For future

work it would be interesting to demonstrate model's accuracy trained with more epochs and different learning parameters to see how *cross-entropy validation loss* changes also if local minima changes notably.

REFERENCES

- [1] Action recognition on hmdb-51. <https://paperswithcode.com/sota/action-recognition-in-videos-on-hmdb-51>.
- [2] Hmdb51. <https://paperswithcode.com/dataset/hmdb51>.
- [3] E. Garrote T. Poggio H. Kuehne, H. Jhuang and T. Serre. A large video database for human motion recognition. *ICCV*, 2011.
- [4] Dan Kondratyuk, Liangzhe Yuan, Yandong Li, Li Zhang, Mingxing Tan, Matthew Brown, and Boqing Gong. Movinets: Mobile video networks for efficient video recognition. *arXiv preprint arXiv:2103.11511*, 2021.
- [5] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 2016.
- [6] Pulkit Sharma. Step by step deep learning tutorial to build your own video classification model. <https://www.analyticsvidhya.com/blog/2019/09/step-by-step-deep-learning-tutorial-video-classification-python/>, 2019.
- [7] Karen Simonyan and Andrew Zisserman. Two-stream convolutional networks for action recognition in videos. *arXiv preprint arXiv:1406.2199*, 2014.

APPENDIX

STAGE	OPERATION	OUTPUT SIZE
data	stride 5, RGB	50×172^2
conv ₁	$1 \times 3^2, 8$	50×86^2
block ₂	$\begin{bmatrix} 1 \times 5^2, 8, 40 \end{bmatrix}$	50×43^2
block ₃	$\begin{bmatrix} 5 \times 3^2, 32, 80 \\ 3 \times 3^2, 32, 80 \\ 3 \times 3^2, 32, 80 \end{bmatrix}$	50×21^2
block ₄	$\begin{bmatrix} 5 \times 3^2, 56, 184 \\ 3 \times 3^2, 56, 112 \\ 3 \times 3^2, 56, 184 \end{bmatrix}$	50×10^2
block ₅	$\begin{bmatrix} 5 \times 3^2, 56, 184 \\ 3 \times 3^2, 56, 184 \\ 3 \times 3^2, 56, 184 \\ 3 \times 3^2, 56, 184 \end{bmatrix}$	50×10^2
block ₆	$\begin{bmatrix} 5 \times 3^2, 104, 344 \\ 1 \times 5^2, 104, 280 \\ 1 \times 5^2, 104, 280 \\ 1 \times 5^2, 104, 344 \end{bmatrix}$	50×5^2
conv ₇	$1 \times 1^2, 480$	50×5^2
pool ₈	50×5^2	1×1^2
dense ₉	$1 \times 1^2, 2048$	1×1^2
dense ₁₀	$1 \times 1^2, 600$	1×1^2

Figure 10 – *MoViNet-A0*

STAGE	OPERATION	OUTPUT SIZE
data	stride 5, RGB	50×172^2
conv ₁	$1 \times 3^2, 16$	50×86^2
block ₂	$\begin{bmatrix} 1 \times 5^2, 16, 40 \\ 3 \times 3^2, 16, 40 \end{bmatrix}$	50×43^2
block ₃	$\begin{bmatrix} 3 \times 3^2, 40, 96 \\ 3 \times 3^2, 40, 120 \\ 3 \times 3^2, 40, 96 \\ 3 \times 3^2, 40, 96 \end{bmatrix}$	50×21^2
block ₄	$\begin{bmatrix} 5 \times 3^2, 64, 216 \\ 3 \times 3^2, 64, 128 \\ 3 \times 3^2, 64, 216 \\ 3 \times 3^2, 64, 168 \\ 3 \times 3^2, 64, 216 \end{bmatrix}$	50×10^2
block ₅	$\begin{bmatrix} 5 \times 3^2, 64, 216 \\ 3 \times 3^2, 64, 216 \\ 3 \times 3^2, 64, 216 \\ 3 \times 3^2, 64, 128 \\ 1 \times 5^2, 64, 128 \\ 3 \times 3^2, 64, 216 \end{bmatrix}$	50×10^2
block ₆	$\begin{bmatrix} 5 \times 3^2, 136, 456 \\ 1 \times 5^2, 136, 360 \\ 1 \times 5^2, 136, 360 \\ 1 \times 5^2, 136, 456 \\ 3 \times 3^2, 136, 456 \\ 1 \times 3^2, 136, 544 \end{bmatrix}$	50×5^2
conv ₇	$1 \times 1^2, 600$	50×5^2
pool ₈	50×5^2	1×1^2
dense ₉	$1 \times 1^2, 2048$	1×1^2
dense ₁₀	$1 \times 1^2, 600$	1×1^2

Figure 11 – *MoViNet-A1*

STAGE	OPERATION	OUTPUT SIZE
data	stride 5, RGB	50×224^2
conv ₁	$1 \times 3^2, 16$	50×112^2
block ₂	$\begin{bmatrix} 1 \times 5^2, 16, 40 \\ 3 \times 3^2, 16, 40 \\ 3 \times 3^2, 16, 64 \end{bmatrix}$	50×56^2
block ₃	$\begin{bmatrix} 3 \times 3^2, 40, 96 \\ 3 \times 3^2, 40, 120 \\ 3 \times 3^2, 40, 96 \\ 3 \times 3^2, 40, 96 \\ 3 \times 3^2, 40, 120 \end{bmatrix}$	50×28^2
block ₄	$\begin{bmatrix} 5 \times 3^2, 72, 240 \\ 3 \times 3^2, 72, 160 \\ 3 \times 3^2, 72, 240 \\ 3 \times 3^2, 72, 192 \\ 3 \times 3^2, 72, 240 \end{bmatrix}$	50×14^2
block ₅	$\begin{bmatrix} 5 \times 3^2, 72, 240 \\ 3 \times 3^2, 72, 240 \\ 3 \times 3^2, 72, 240 \\ 3 \times 3^2, 72, 240 \\ 1 \times 5^2, 72, 144 \\ 3 \times 3^2, 72, 240 \end{bmatrix}$	50×14^2
block ₆	$\begin{bmatrix} 5 \times 3^2, 144, 480 \\ 1 \times 5^2, 144, 384 \\ 1 \times 5^2, 144, 384 \\ 1 \times 5^2, 144, 480 \\ 1 \times 5^2, 144, 480 \\ 3 \times 3^2, 144, 480 \\ 1 \times 3^2, 144, 576 \end{bmatrix}$	50×7^2
conv ₇	$1 \times 1^2, 640$	50×7^2
pool ₈	50×7^2	1×1^2
dense ₉	$1 \times 1^2, 2048$	1×1^2
dense ₁₀	$1 \times 1^2, 600$	1×1^2

Figure 12 – *MoViNet-A2*